QRAF: A High-Performance Local LLM Inference Runtime with a Custom Binary Model Format Optimized for Apple Silicon

Technical White Paper

Simon-Pierre Boucher
Université Laval, Québec, QC, Canada
spbou4@ulaval.ca
https://github.com/simonpierreboucher02/qraf

April 2026

Abstract

We present **QRAF** (Quantized Runtime Artifact Format), a complete local large language model (LLM) inference runtime implemented from scratch in modern C++17. QRAF introduces a novel binary model format designed for memory-mapped, zero-copy tensor access with native quantization support, paired with an inference engine optimized for Apple Silicon processors (M1–M5). The system achieves a **41× speedup** over a scalar baseline through Apple Accelerate framework integration, ARM NEON SIMD kernels, and Grand Central Dispatch (GCD) multi-threading. QRAF supports 7 model architectures, includes a universal model converter supporting HuggingFace, GGUF, safetensors, and PyTorch formats, and implements LoRA adapter merging, KV cache quantization, speculative decoding, and continuous batching. We provide formal analysis of computational complexity, quantization error bounds, and throughput models. Evaluation on 26 models (70M–3B parameters) demonstrates inference speeds from 8.5 to 512 tokens per second on consumer hardware. All source code is released under the MIT license.
Keywords: LLM inference, model format, quantization, Apple Silicon, NEON SIMD, Metal GPU, transformer, local AI

1 Introduction

The deployment of large language models (LLMs) has traditionally required cloud infrastructure with specialized accelerators [3, 25]. However, growing concerns about privacy, latency, cost, and offline availability have driven significant interest in *local inference* — running LLMs directly on consumer hardware [5, 6].

Existing local inference solutions such as llama.cpp [15], Ollama [18], and vLLM [12] face limitations in format interoperability, architecture coupling, platform-specific optimization, and deployment complexity.

We address these limitations with QRAF, which contributes:

1. A novel binary model format (.qraf) with mmap-first, zero-copy design and 64-byte SIMD alignment (Section 6).
2. A multi-architecture inference engine supporting 7 transformer families (Section 7).
3. Apple Silicon optimizations achieving 41× speedup (Section 8).
4. Formal computational and quantization error analysis (Sections 4, 5).
5. Advanced features: LoRA, KV quantization, speculative decoding, continuous batching (Section 9).
6. Evaluation on 26 models from 70M to 3B parameters (Section 10).

2 Related Work

Local Inference Engines. llama.cpp [15] pioneered efficient CPU inference with 4-bit quantization and GGUF format. Ollama [18] provides user-friendly model management atop llama.cpp. vLLM [12] introduced PagedAttention for GPU

memory efficiency. MLC-LLM [16] uses TVM compilation for cross-platform kernels.

Model Formats. GGUF [7] stores quantized tensors with key-value metadata. safetensors [20] provides safe tensor storage without Python pickle. Neither embeds complete tokenizer data with BPE merge tables.

Quantization. GPTQ [6] uses approximate Hessian information for optimal rounding. AWQ [14] protects salient weights. SqueezeLLM [11] introduces non-uniform quantization.

QRAF differentiates by introducing a purpose-built format with embedded tokenizer, fully exploiting Apple Silicon's heterogeneous compute, and providing end-to-end tooling.

3 Mathematical Foundations

3.1 Transformer Architecture

Definition 1 (Decoder-Only Transformer)**.** *A decoder-only transformer with L layers, hidden dimension d, H attention heads (head dimension $d_h = d/H$), H_{kv} key-value heads, and intermediate dimension d_{ff} defines the mapping $f : \mathbb{N}^T \to \mathbb{R}^{T\times V}$ as:*

$$f(\boldsymbol{x}) = \boldsymbol{W}_{head} \cdot Norm\left(\boldsymbol{h}^{(L)}\right) \tag{1}$$

where $\boldsymbol{h}^{(L)}$ is the output of the L-th layer.

3.2 Multi-Head Attention

Given input $\boldsymbol{x} \in \mathbb{R}^d$ at position t, the attention mechanism computes:

$$\boldsymbol{q}_h = \boldsymbol{W}_Q^{(h)}\boldsymbol{x} + \boldsymbol{b}_Q^{(h)}, \quad h = 1, \ldots, H \tag{2}$$

$$\boldsymbol{k}_g = \boldsymbol{W}_K^{(g)}\boldsymbol{x} + \boldsymbol{b}_K^{(g)}, \quad g = 1, \ldots, H_{kv} \tag{3}$$

$$\boldsymbol{v}_g = \boldsymbol{W}_V^{(g)}\boldsymbol{x} + \boldsymbol{b}_V^{(g)} \tag{4}$$

where $\boldsymbol{W}_Q^{(h)} \in \mathbb{R}^{d_h\times d}$, $\boldsymbol{W}_K^{(g)}, \boldsymbol{W}_V^{(g)} \in \mathbb{R}^{d_h\times d}$, and bias terms $\boldsymbol{b}$ are optional (present in Qwen2, GPT-2; absent in LLaMA).

Definition 2 (Grouped Query Attention (GQA))**.** *In GQA [1], each key-value head g is shared by $r = H/H_{kv}$ query heads. The attention for query head h using KV head $g = \lfloor h/r \rfloor$ is:*

$$\alpha_{h,t,s} = \frac{\exp\left(\boldsymbol{q}_{h,t}^\top \boldsymbol{k}_{g,s}/\sqrt{d_h}\right)}{\sum_{s'=1}^{t}\exp\left(\boldsymbol{q}_{h,t}^\top \boldsymbol{k}_{g,s'}/\sqrt{d_h}\right)} \tag{5}$$

$$\boldsymbol{o}_{h,t} = \sum_{s=1}^{t} \alpha_{h,t,s} \cdot \boldsymbol{v}_{g,s} \tag{6}$$

Remark 1. *When $H_{kv} = H$, GQA reduces to standard Multi-Head Attention (MHA). When $H_{kv} = 1$, it becomes Multi-Query Attention (MQA) [22]. The KV cache memory scales as $O(L \cdot H_{kv} \cdot d_h \cdot T)$ rather than $O(L \cdot H \cdot d_h \cdot T)$, providing significant savings when $H_{kv} \ll H$.*

The concatenated multi-head output is projected:

$$\mathrm{Attn}(\boldsymbol{x}, t) = \boldsymbol{W}_O\left[\boldsymbol{o}_{1,t}; \boldsymbol{o}_{2,t}; \ldots; \boldsymbol{o}_{H,t}\right] \tag{7}$$

3.3 Rotary Position Embedding (RoPE)

Definition 3 (RoPE [24])**.** *For position t and dimension pair index $i \in \{0, \ldots, d_h/2 - 1\}$, define the rotation matrix $\boldsymbol{R}_t^{(i)} \in \mathbb{R}^{2\times 2}$:*

$$\boldsymbol{R}_t^{(i)} = \begin{pmatrix} \cos(t\theta_i) & -\sin(t\theta_i) \\ \sin(t\theta_i) & \cos(t\theta_i) \end{pmatrix} \tag{8}$$

where the frequency is:

$$\theta_i = \theta_{base}^{-2i/d_h} \tag{9}$$

The full RoPE transformation is the block-diagonal rotation:

$$\mathrm{RoPE}(\boldsymbol{q}, t) = \mathrm{diag}\left(\boldsymbol{R}_t^{(0)}, \boldsymbol{R}_t^{(1)}, \ldots, \boldsymbol{R}_t^{(d_h/2-1)}\right)\boldsymbol{q} \tag{10}$$

Proposition 1 (RoPE Relative Position Property)**.** *For queries $\boldsymbol{q}_m = \mathrm{RoPE}(\boldsymbol{q}, m)$ and keys $\boldsymbol{k}_n = \mathrm{RoPE}(\boldsymbol{k}, n)$:*

$$\boldsymbol{q}_m^\top \boldsymbol{k}_n = \boldsymbol{q}^\top \boldsymbol{R}_{n-m}\boldsymbol{k} \tag{11}$$

where $\boldsymbol{R}_{n-m} = \mathrm{diag}\left(\boldsymbol{R}_{n-m}^{(0)}, \ldots, \boldsymbol{R}_{n-m}^{(d_h/2-1)}\right)$. Thus the dot product depends only on the relative position $n - m$.

Proof. By the orthogonality of rotation matrices: $(\boldsymbol{R}_m)^\top \boldsymbol{R}_n = \boldsymbol{R}_{n-m}$. Since each 2×2 block is an orthogonal rotation, $(\boldsymbol{R}_m^{(i)})^\top \boldsymbol{R}_n^{(i)} = \boldsymbol{R}_{n-m}^{(i)}$, and the block-diagonal structure preserves this property. □

3.4 Normalization

Definition 4 (RMSNorm [27])**.** *For input $\boldsymbol{x} \in \mathbb{R}^d$ with learnable weight $\boldsymbol{\gamma} \in \mathbb{R}^d$:*

$$\mathrm{RMSNorm}(\boldsymbol{x}) = \frac{\boldsymbol{x}}{RMS(\boldsymbol{x})} \odot \boldsymbol{\gamma}, \quad RMS(\boldsymbol{x}) = \sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon} \tag{12}$$

Definition 5 (LayerNorm [2]).

$$\text{LayerNorm}(\boldsymbol{x}) = \frac{\boldsymbol{x} - \mu}{\sigma} \odot \boldsymbol{\gamma} + \boldsymbol{\beta} \tag{13}$$

where $\mu = \frac{1}{d}\sum_i x_i$, $\sigma = \sqrt{\frac{1}{d}\sum_i (x_i - \mu)^2 + \epsilon}$, *and* $\boldsymbol{\beta}$ *is a learnable bias.*

Remark 2. *RMSNorm requires* $2d$ *FLOPs (one pass for sum of squares, one for scaling), while LayerNorm requires* $4d$ *FLOPs (two passes for mean and variance, two for normalization and affine). QRAF dispatches between them based on the architecture type.*

3.5 Feed-Forward Networks

Definition 6 (SwiGLU MLP [23]). *The gated linear unit variant used in LLaMA/Qwen:*

$$FFN_{SwiGLU}(\boldsymbol{x}) = \boldsymbol{W}_{down}\left(\text{SiLU}(\boldsymbol{W}_{gate}\boldsymbol{x}) \odot \boldsymbol{W}_{up}\boldsymbol{x}\right) \tag{14}$$

where $\boldsymbol{W}_{gate}, \boldsymbol{W}_{up} \in \mathbb{R}^{d_{ff} \times d}$, $\boldsymbol{W}_{down} \in \mathbb{R}^{d \times d_{ff}}$, *and* $\text{SiLU}(x) = x \cdot \sigma(x)$ *with* σ *being the sigmoid function.*

Definition 7 (Standard MLP). *Used in GPT-2/OPT/Pythia:*

$$FFN_{std}(\boldsymbol{x}) = \boldsymbol{W}_2 \cdot \phi(\boldsymbol{W}_1\boldsymbol{x} + \boldsymbol{b}_1) + \boldsymbol{b}_2 \tag{15}$$

where ϕ *is the activation function (GELU for GPT-2/Pythia, ReLU for OPT):*

$$\text{GELU}(x) = \frac{x}{2}\left(1 + erf\left(\frac{x}{\sqrt{2}}\right)\right) \approx \frac{x}{2}\left(1 + \tanh\left(\sqrt{\frac{2}{\pi}}(x + 0.044715x^3)\right)\right) \tag{16}$$

3.6 Sampling Theory

Definition 8 (Temperature-scaled Categorical Sampling). *Given logits* $\boldsymbol{z} \in \mathbb{R}^V$ *and temperature* $\tau > 0$*:*

$$p(x = i) = \frac{\exp(z_i/\tau)}{\sum_{j=1}^{V}\exp(z_j/\tau)} \tag{17}$$

Definition 9 (Top-k Filtering). *Let* $\mathcal{S}_k \subseteq \{1, \ldots, V\}$ *be the indices of the* k *largest logits. The filtered distribution is:*

$$p_{top\text{-}k}(x = i) = \begin{cases} \frac{p(x=i)}{\sum_{j \in \mathcal{S}_k} p(x=j)} & \text{if } i \in \mathcal{S}_k \\ 0 & \text{otherwise} \end{cases} \tag{18}$$

Definition 10 (Nucleus (Top-p) Sampling [8]). *Let* π *be a permutation sorting probabilities in descending order. The nucleus* $\mathcal{N}_p$ *is:*

$$\mathcal{N}_p = \left\{ i : \sum_{j : p(\pi(j)) \geq p(i)} p(\pi(j)) \leq p \right\} \tag{19}$$

The distribution is renormalized over $\mathcal{N}_p$.

Definition 11 (Repetition Penalty [10]). *For token* i *appearing in the recent context* $\mathcal{C}$*:*

$$z_i' = \begin{cases} z_i/\rho & \text{if } z_i > 0 \text{ and } i \in \mathcal{C} \\ z_i \cdot \rho & \text{if } z_i \leq 0 \text{ and } i \in \mathcal{C} \\ z_i & \text{otherwise} \end{cases} \tag{20}$$

where $\rho > 1$ *is the penalty factor.*

4 Computational Complexity Analysis

4.1 Per-Token Complexity

Theorem 1 (Single-Token Forward Pass Complexity). *For a decoder-only transformer with* L *layers, hidden dimension* d*,* H *query heads,* H_{kv} *KV heads, head dimension* $d_h = d/H$*, intermediate dimension* d_{ff}*, vocabulary size* V*, and current sequence length* T*, the floating-point operations for generating one token are:*

$$\mathcal{F}(T) = L \cdot \mathcal{F}_{layer}(T) + \mathcal{F}_{head} \tag{21}$$

where:

$$\begin{aligned} \mathcal{F}_{layer}(T) = & \underbrace{2d(H \cdot d_h + 2H_{kv} \cdot d_h + d)}_{\text{QKV + output projections}} \\ & + \underbrace{2H \cdot d_h \cdot T}_{\text{attention scores}} + \underbrace{2H \cdot d_h \cdot T}_{\text{value aggregation}} \\ & + \underbrace{\mathcal{F}_{MLP}}_{\text{feed-forward}} + \underbrace{O(d)}_{\text{norms, residuals}} \end{aligned} \tag{22}$$

For SwiGLU: $\mathcal{F}_{MLP} = 6d \cdot d_{ff}$ *(three projections). For standard MLP:* $\mathcal{F}_{MLP} = 4d \cdot d_{ff}$ *(two projections).*
The output projection contributes: $\mathcal{F}_{head} = 2Vd$.

Proof. Each matrix-vector product $\boldsymbol{y} = \boldsymbol{W}\boldsymbol{x}$ with $\boldsymbol{W} \in \mathbb{R}^{m \times n}$ requires $2mn$ FLOPs (n multiplications and $n - 1$ additions per output element, $\approx 2mn$ total). The QKV projections contribute $2d \cdot Hd_h$ (Q) + $2d \cdot H_{kv}d_h$ (K) + $2d \cdot H_{kv}d_h$ (V). Output projection: $2d \cdot Hd_h = 2d^2$. Attention scores: H heads $\times$ T positions $\times$ $2d_h$ FLOPs per dot product. Value aggregation: identical to scores. SwiGLU has three weight matrices of size $d_{ff} \times d$ or $d \times d_{ff}$. □

Proposition 2 (Arithmetic Intensity). *The arithmetic intensity (FLOPs per byte of memory accessed) for autoregressive token generation is:*

$$I = \frac{\mathcal{F}(T)}{B_{weights} + B_{KV}(T)} \tag{23}$$

where $B_{weights} = \sum_{all\ tensors} |\boldsymbol{W}| \cdot b_w$ is total weight bytes (accessed once per token via mmap), b_w is bytes per weight element, and:

$$B_{KV}(T) = 2L \cdot H_{kv} \cdot d_h \cdot T \cdot b_{kv} \tag{24}$$

is KV cache bytes read. For autoregressive generation, $I < 1$ FLOP/byte for most practical configurations, making inference ***memory-bandwidth bound****.*

Remark 3. *Since inference is memory-bandwidth bound (Proposition 2), the primary optimization target is minimizing memory traffic. This motivates: (1) weight quantization to reduce b_w, (2) GQA/MQA to reduce B_{KV}, (3) mmap to leverage OS page cache, and (4) KV cache quantization to reduce b_{kv}.*

4.2 Memory Requirements

Theorem 2 (Model Memory Footprint)**.** *The total memory for a model with P parameters at b_w bits per weight, with KV cache at b_{kv} bits per element, sequence length T:*

$$M = \underbrace{P \cdot \frac{b_w}{8}}_{weights} + \underbrace{2L \cdot H_{kv} \cdot d_h \cdot T \cdot \frac{b_{kv}}{8}}_{KV\ cache} + \underbrace{O(d + d_{ff})}_{activations} \tag{25}$$

For the Qwen2.5-3B model ($P = 3\times10^9$, $L = 36$, $H_{kv} = 2$, $d_h = 128$, $T = 2048$):

$$M_{\text{weights}} = 3 \times 10^9 \times 4 = 12 \text{ GB (f32)} \tag{26}$$

$$M_{\text{KV}} = 2 \times 36 \times 2 \times 128 \times 2048 \times 4 = 151 \text{ MB} \tag{27}$$

$$M_{\text{total}} \approx 12.15 \text{ GB} \tag{28}$$

With int8 KV cache: $M_{\text{KV}} = 37.7$ MB (4× reduction).

5 Quantization Theory

5.1 Block-wise Symmetric Quantization

Definition 12 (Qb_0 Quantization)**.** *For a block of B floating-point values $\boldsymbol{x} = (x_1, \ldots, x_B)$, b-bit symmetric quantization computes:*

$$s = \frac{\max_i |x_i|}{2^{b-1} - 1}, \quad \hat{x}_i = round\left(\frac{x_i}{s}\right) \tag{29}$$

The dequantized value is $\tilde{x}_i = s \cdot \hat{x}_i$.

Theorem 3 (Quantization Error Bound)**.** *For b-bit symmetric quantization with block size B, the per-element quantization error is bounded by:*

$$|x_i - \tilde{x}_i| \le \frac{s}{2} = \frac{\|\boldsymbol{x}\|_\infty}{2(2^{b-1} - 1)} \tag{30}$$

and the root-mean-square error over a block satisfies:

$$RMSE = \sqrt{\frac{1}{B}\sum_{i=1}^{B}(x_i - \tilde{x}_i)^2} \le \frac{\|\boldsymbol{x}\|_\infty}{2(2^{b-1} - 1)} \tag{31}$$

Proof. Each rounding operation introduces error $|x_i - s \cdot \text{round}(x_i/s)| \le s/2$. With $s = \|\boldsymbol{x}\|_\infty/(2^{b-1} - 1)$, we get the first bound. The RMSE bound follows from RMSE $\le \max_i |e_i|$. □

Table 1: Quantization error bounds by bit width.

Scheme	b	Max error	Ratio	Size ratio
Q4_0	4	$\|\boldsymbol{x}\|_\infty/14$	7.1%	1/8
Q8_0	8	$\|\boldsymbol{x}\|_\infty/254$	0.39%	1/4
FP16	16	$\sim 10^{-3}\|\boldsymbol{x}\|_\infty$	0.1%	1/2
FP32	32	0	0%	1

5.2 Quantized Dot Product

Proposition 3 (Error of Quantized Dot Product)**.** *For vectors $\boldsymbol{w}, \boldsymbol{x} \in \mathbb{R}^n$ where $\boldsymbol{w}$ is quantized to $\tilde{\boldsymbol{w}}$:*

$$|\boldsymbol{w}^\top \boldsymbol{x} - \tilde{\boldsymbol{w}}^\top \boldsymbol{x}| = |(\boldsymbol{w} - \tilde{\boldsymbol{w}})^\top \boldsymbol{x}| \le \|\boldsymbol{w} - \tilde{\boldsymbol{w}}\|_2 \cdot \|\boldsymbol{x}\|_2 \tag{32}$$

With block-wise Q8_0 quantization (n/B blocks):

$$|\boldsymbol{w}^\top \boldsymbol{x} - \tilde{\boldsymbol{w}}^\top \boldsymbol{x}| \le \frac{\sqrt{n}}{2 \cdot 127} \max_b \|\boldsymbol{w}_b\|_\infty \cdot \|\boldsymbol{x}\|_2 \tag{33}$$

5.3 KV Cache Quantization

We quantize the KV cache to int8 per-head per-position:

$$\hat{\boldsymbol{k}}_{l,t,h} = \text{round}\left(\frac{127 \cdot \boldsymbol{k}_{l,t,h}}{\|\boldsymbol{k}_{l,t,h}\|_\infty}\right), \quad s^{(k)}_{l,t,h} = \frac{\|\boldsymbol{k}_{l,t,h}\|_\infty}{127} \tag{34}$$

The attention score with quantized keys:

$$\boldsymbol{q}_t^\top \tilde{\boldsymbol{k}}_s = s_s^{(k)} \cdot \boldsymbol{q}_t^\top \hat{\boldsymbol{k}}_s \tag{35}$$

where the inner product $\boldsymbol{q}_t^\top \hat{\boldsymbol{k}}_s$ can be computed efficiently using NEON int8 widening instructions.

Theorem 4 (KV Cache Memory Reduction)**.** *For L layers, H_{kv} KV heads, head dimension d_h, and sequence length T:*

$$M_{KV}^{f32} = 2LH_{kv}d_hT \cdot 4 \text{ bytes} \tag{36}$$

$$M_{KV}^{int8} = 2LH_{kv}d_hT \cdot 1 + 2LH_{kv}T \cdot 4 \text{ bytes (scales)} \tag{37}$$

$$Ratio = \frac{M_{KV}^{int8}}{M_{KV}^{f32}} = \frac{1}{4} + \frac{1}{d_h} \approx \frac{1}{4} \text{ for } d_h \gg 4 \tag{38}$$

6 QRAF Binary Format

The QRAF format consists of seven contiguous sections with 64-byte aligned tensor data:

Table 2: QRAF file sections.

Section	Description	Size
Header	Magic, offsets	128 B
Config	Key-value params	Var.
Tokenizer	Vocab + BPE merges	Var.
Tensor Dir	$80 \times N_T$	Fixed
Quant Dir	$32 \times N_Q$	Fixed
String Table	UTF-8 strings	Var.
Data Blocks	64B-aligned weights	Var.

Tensor lookup uses FNV-1a hashing [Noll & Fowler]:

$$h_0 = 14695981039346656037, \quad h_i = (h_{i-1} \oplus c_i) \times 1099511628211 \tag{39}$$

The tokenizer section stores the complete BPE merge table (50,000–151,000 entries), each as $(a_{\text{id}}, b_{\text{id}}, r_{\text{id}}, p)$ where p is the merge priority. This ensures tokenization correctness matching the original HuggingFace tokenizer.

7 Inference Engine

7.1 Architecture Dispatch

The engine supports 7 architectures through a compile-time dispatch based on the `arch_type` enum:

Algorithm 1 Multi-Architecture Forward Pass

Require: token x_t, position t, architecture $\mathcal{A}$

$\boldsymbol{h} \leftarrow \boldsymbol{E}[x_t]$ ▷ Token embedding
if $\mathcal{A} \in \{\text{GPT-2}, \text{OPT}, \text{StarCoder}\}$ **then**
  $\boldsymbol{h} \leftarrow \boldsymbol{h} + \boldsymbol{P}[t]$ ▷ Learned positions
end if
if $\mathcal{A} = \text{OPT}$ and project_in exists **then**
  $\boldsymbol{h} \leftarrow \boldsymbol{W}_{\text{proj_in}}\boldsymbol{h}$ ▷ OPT embed projection
end if
for $l = 0$ to $L - 1$ **do**
  $\boldsymbol{r} \leftarrow \boldsymbol{h}$
  if $\mathcal{A}$.parallel_attn **then** ▷ GPT-NeoX, CodeGen
    $\boldsymbol{a} \leftarrow \text{Attn}(\text{Norm}(\boldsymbol{h}))$
    $\boldsymbol{m} \leftarrow \text{MLP}(\text{Norm}(\boldsymbol{h}))$
    $\boldsymbol{h} \leftarrow \boldsymbol{r} + \boldsymbol{a} + \boldsymbol{m}$
  else ▷ LLaMA, GPT-2, OPT
    $\boldsymbol{h} \leftarrow \boldsymbol{r} + \text{Attn}(\text{Norm}(\boldsymbol{h}))$
    $\boldsymbol{r} \leftarrow \boldsymbol{h}$
    $\boldsymbol{h} \leftarrow \boldsymbol{r} + \text{MLP}(\text{Norm}(\boldsymbol{h}))$
  end if
end for
return $\boldsymbol{W}_{\text{head}} \cdot \text{Norm}(\boldsymbol{h})$

7.2 BPE Tokenization

Algorithm 2 Byte-Pair Encoding with Merge Table

Require: text s, vocabulary V, merge table $\mathcal{M}$

$\boldsymbol{u} \leftarrow \text{ByteToUnicode}(s)$ ▷ GPT-2 byte mapping
$\boldsymbol{t} \leftarrow [\,]$ ▷ Initialize with characters
for each UTF-8 character c in $\boldsymbol{u}$ **do**
  $\boldsymbol{t}.\text{append}(V[c])$
end for
while $|\boldsymbol{t}| > 1$ **do**
  $(i^*, p^*) \leftarrow \arg\max_i \{p : (t_i, t_{i+1}, r, p) \in \mathcal{M}\}$
  if $p^* = -\infty$ **then break**
  end if
  Replace t_{i^*}, t_{i^*+1} with merged token r
end while
return $\boldsymbol{t}$

8 Apple Silicon Optimization

8.1 Optimization Hierarchy

Table 3: Backend selection hierarchy.

Operation	Backend	Technology
f32 MatVec	Accelerate	`cblas_sgemv`
Softmax	Accelerate	`vvexpf` + `vDSP_maxv`
SiLU	Accelerate	`vvexpf` + `vDSP_vdiv`
RMSNorm	NEON	`vfmaq_f32`
Q8 Dot	NEON	int8→f32 widening
Q4 Dot	NEON	nibble unpack
RoPE	NEON	`vld2q/vst2q`
GPU ops	Metal	Compute shaders
Parallel rows	GCD	`dispatch_apply`

8.2 NEON Throughput Model

Theorem 5 (NEON Dot Product Throughput)**.** *With 4 accumulator registers and 16-wide unrolling, processing n elements:*

$$T_{NEON}(n) = \left\lceil \frac{n}{16} \right\rceil \cdot t_{FMA} + t_{reduce} \tag{40}$$

where $t_{FMA} \approx 1$ cycle (Apple Silicon has 2 FMA units per core with 4-cycle latency, fully pipelined) and $t_{reduce} \approx 4$ cycles for horizontal sum via `vaddvq_f32`.

For the scalar baseline: $T_{scalar}(n) = 2n$ cycles (multiply + add per element). The theoretical speedup is:

$$S = \frac{T_{scalar}}{T_{NEON}} = \frac{2n}{\lceil n/16 \rceil + 4} \xrightarrow{n\to\infty} 32 \tag{41}$$

8.3 GCD Threading Model

For a matrix-vector product $\boldsymbol{y} = \boldsymbol{W}\boldsymbol{x}$ with $\boldsymbol{W} \in \mathbb{R}^{m\times n}$, parallelized over P threads:

$$T_{\text{threaded}} = \frac{m}{P} \cdot T_{\text{dot}}(n) + T_{\text{overhead}} \tag{42}$$

where T_{overhead} is GCD dispatch overhead ($\approx 2\mu$s). Threading is beneficial when:

$$\frac{m}{P} \cdot T_{\text{dot}}(n) > T_{\text{overhead}} \implies m > \frac{P \cdot T_{\text{overhead}}}{T_{\text{dot}}(n)} \tag{43}$$

For $n = 2048$, $T_{\text{dot}} \approx 0.5\mu$s, $P = 8$: threading is beneficial when $m > 32$, satisfied by all practical layer dimensions.

9 Advanced Features

9.1 LoRA Adapter Merging

Definition 13 (Low-Rank Adaptation [9])**.** *Given pretrained $\boldsymbol{W} \in \mathbb{R}^{d_{out}\times d_{in}}$ and low-rank matrices $\boldsymbol{A} \in \mathbb{R}^{r\times d_{in}}$, $\boldsymbol{B} \in \mathbb{R}^{d_{out}\times r}$ with rank $r \ll \min(d_{in}, d_{out})$:*

$$\boldsymbol{W}' = \boldsymbol{W} + \frac{\alpha}{r}\boldsymbol{B}\boldsymbol{A} \tag{44}$$

Proposition 4 (LoRA Merge Complexity)**.** *Merging one weight matrix costs $O(d_{out} \cdot r \cdot d_{in})$ FLOPs. For $r = 16$ and $d = 2048$, this is $\sim 134M$ FLOPs per matrix, negligible compared to a single forward pass.*

9.2 Speculative Decoding

Theorem 6 (Speculative Decoding Speedup [13])**.** *Let α be the acceptance rate of draft tokens. With K speculative tokens per step and draft model latency t_d vs. main model latency t_m:*

$$Speedup = \frac{(1 + K\alpha) \cdot t_m}{t_m + K \cdot t_d} \tag{45}$$

If $t_d \ll t_m$ and α is high, the speedup approaches $1 + K\alpha$.

The rejection sampling criterion ensures correctness:

$$P(\text{accept } x_i) = \min\left(1, \frac{p_{\text{main}}(x_i)}{p_{\text{draft}}(x_i)}\right) \tag{46}$$

Rejected tokens are resampled from:

$$p_{\text{adj}}(x) = \frac{\max(0, p_{\text{main}}(x) - p_{\text{draft}}(x))}{Z} \tag{47}$$

where $Z = \sum_x \max(0, p_{\text{main}}(x) - p_{\text{draft}}(x))$.

Proposition 5 (Correctness)**.** *The combined distribution of accepted and resampled tokens is exactly p_{main}, regardless of the draft model quality.*

9.3 Continuous Batching

Definition 14 (Iteration-Level Scheduling)**.** *At each decode step t, the scheduler maintains a set of active requests $\mathcal{A}_t$ with per-request KV caches. New requests enter from queue $\mathcal{Q}$:*

$$\mathcal{A}_{t+1} = (\mathcal{A}_t \backslash \mathcal{D}_t) \cup \{q \in \mathcal{Q} : |\mathcal{A}_t \backslash \mathcal{D}_t| < B_{\max}\} \tag{48}$$

where $\mathcal{D}_t$ are completed/cancelled requests and $B_{\max}$ is the maximum batch size.

10 Evaluation

10.1 Experimental Setup

All experiments: Apple MacBook Pro, Apple Silicon ARM64, 18 hardware threads. Compiled with `-O3 -mcpu=apple-m1` (Apple Clang 21.0). Models use f32 weights.

10.2 Inference Speed

Table 4: Inference speed across representative models.

Model	Arch	Params	tok/s	ms/tok
Pythia-70M	NeoX	70M	512.1	2.0
DistilGPT2	GPT-2	82M	281.3	3.6
GPT-2	GPT-2	124M	178.8	5.6
SmolLM2-135M	LLaMA	135M	172.6	5.8
Pythia-410M	NeoX	410M	60.6	16.5
Qwen2.5-0.5B-I	Qwen2	494M	23.0	43.5
SmolLM2-1.7B-I	LLaMA	1.7B	9.4	106
Qwen2.5-3B-I	**Qwen2**	**3B**	**8.5**	**112**

10.3 Optimization Impact

Table 5: Cumulative speedup breakdown.

Configuration	tok/s	Speedup
Scalar baseline	5.5	1.0×
+ Accelerate (`cblas_sgemv`)	44.0	8.0×
+ NEON (element-wise)	52.8	9.6×
+ GCD (12 threads)	58.1	10.6×
Full (Accel. + NEON + GCD)	226.0	**41.1**×

10.4 Scaling Analysis

We fit a throughput model to our measurements. Let P denote parameter count in billions:

$$\text{tok/s} \approx \frac{C}{P^{\beta}} \tag{49}$$

Fitting to our data yields $C \approx 28.5$ and $\beta \approx 0.82$ ($R^2 = 0.94$), indicating sub-linear scaling as expected from the memory-bandwidth bound regime.

10.5 Tokenizer Quality Impact

Table 6: Effect of BPE merges on tokenization correctness (Qwen2.5-0.5B).

	Without merges	With merge
Tokens for "Hello world"	10	2
Matches HuggingFace	No	Yes
Output coherence	Incoherent	Coherent
Encoding method	Greedy longest	BPE merge
Merge count stored	0	151,387

10.6 Text Quality

Table 7: Generated text from Qwen2.5-3B-Instruct.

Task	Output
Quantum	"Quantum computing uses the principles of quantum mechanics to perform computations at extremely high speed."
Translation	"La météo est belle aujourd'hui, allons au parc."
Creative	"Once upon a time, in the vast expanse of Cyberspace, there was an enigmatic robot named Zephyr."

11 Discussion

f32 vs. quantized storage. The current converter stores f32 weights, resulting in large files (13 GB for 3B). Weight quantization to Q4 would reduce this to ∼1.6 GB with bounded error (Theorem 3).

Memory-bandwidth bound. By Proposition 2, autoregressive inference has arithmetic intensity < 1 FLOP/byte, meaning GPU utilization is limited by memory bandwidth. Quantization directly addresses this by reducing bytes transferred per operation.

Accelerate vs. custom kernels. For f32 operations, Apple's Accelerate outperforms custom NEON due to microarchitecture-specific tuning. For quantized operations, custom NEON kernels are essential as Accelerate lacks quantized BLAS.

12 Conclusion

We presented QRAF, a complete local LLM inference runtime achieving 41× speedup on Apple

Silicon. Key contributions include a mmap-first binary format, multi-architecture dispatch (7 families, 26 models), formal complexity and error analysis, and advanced features (LoRA, KV quantization, speculative decoding, continuous batching). The system generates coherent text from models up to 3B parameters at 8.5 tokens/second on consumer hardware.

Future work: weight quantization in the converter, Flash Attention, CUDA backend, and support for additional architectures (Phi-3, Gemma, Mamba).

Source code: https://github.com/simonpierreboucher02/qraf (MIT license).

References

[1] Ainslie, J. et al. (2023). GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv:2305.13245*.

[2] Ba, J. L., Kiros, J. R., & Hinton, G. E. (2016). Layer Normalization. *arXiv:1607.06450*.

[3] Brown, T. et al. (2020). Language Models are Few-Shot Learners. *NeurIPS*, 33, 1877–1901.

[4] Chen, C. et al. (2023). Accelerating Large Language Model Decoding with Speculative Sampling. *arXiv:2302.01318*.

[5] Dettmers, T. et al. (2022). GPT3.int8(): 8-bit Matrix Multiplication for Transformers at Scale. *NeurIPS*, 35.

[6] Frantar, E. et al. (2023). GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. *ICLR*.

[7] GGUF Format Specification. (2023). https://github.com/ggerganov/ggml/blob/master/docs/gguf.md

[8] Holtzman, A. et al. (2020). The Curious Case of Neural Text Degeneration. *ICLR*.

[9] Hu, E. J. et al. (2022). LoRA: Low-Rank Adaptation of Large Language Models. *ICLR*.

[10] Keskar, N. S. et al. (2019). CTRL: A Conditional Transformer Language Model for Controllable Generation. *arXiv:1909.05858*.

[11] Kim, S. et al. (2024). SqueezeLLM: Dense-and-Sparse Quantization. *ICML*.

[12] Kwon, W. et al. (2023). Efficient Memory Management for Large Language Model Serving with PagedAttention. *SOSP*.

[13] Leviathan, Y. et al. (2023). Fast Inference from Transformers via Speculative Decoding. *ICML*.

[14] Lin, J. et al. (2024). AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. *MLSys*.

[15] Gerganov, G. (2023). llama.cpp: LLM inference in C/C++. https://github.com/ggerganov/llama.cpp

[16] MLC Team. (2023). MLC-LLM. https://github.com/mlc-ai/mlc-llm

[Noll & Fowler] Noll, L. C. & Fowler, G. FNV Hash. http://www.isthe.com/chongo/tech/comp/fnv/

[18] Ollama. (2024). https://ollama.com

[19] Paszke, A. et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *NeurIPS*, 32.

[20] HuggingFace. (2023). safetensors. https://github.com/huggingface/safetensors

[21] Sennrich, R. et al. (2016). Neural Machine Translation of Rare Words with Subword Units. *ACL*, 1715–1725.

[22] Shazeer, N. (2019). Fast Transformer Decoding: One Write-Head is All You Need. *arXiv:1911.02150*.

[23] Shazeer, N. (2020). GLU Variants Improve Transformer. *arXiv:2002.05202*.

[24] Su, J. et al. (2024). RoFormer: Enhanced Transformer with Rotary Position Embedding. *Neurocomputing*, 568, 127063.

[25] Touvron, H. et al. (2023). LLaMA: Open and Efficient Foundation Language Models. *arXiv:2302.13971*.

[26] Vaswani, A. et al. (2017). Attention Is All You Need. *NeurIPS*, 30.

[27] Zhang, B. & Sennrich, R. (2019). Root Mean Square Layer Normalization. *NeurIPS*.